SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Name : V.V.D.Nagasai Ratan Reddy	**Reg. no. 192324172**
Course Code: DSA0311	**Course Name: Natural Language Processing**
Branch: AI&DS	**Year : 2023**

Assignment 2

1. Figure 4.1 shows the bigram counts from a piece of a bigram grammar from the Berkeley Restaurant Project. Note that the majority of the values are zero. Perform the add-1 smoothed counts for the below bigram probability values. (10M) (CO2) (BLT4)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 4.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences.

2. Suppose someone took all the words in a sentence and reordered them randomly. Write a program which take as input bag of words such a bag of words and produces as output a guess at the original order. You will need to an N-gram grammar produced by your N-gram program (on some corpus. This task is sometimes called bag generation. (10M) (CO2) (BLT4)
3. Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation. Some of these are quite difficult; do your best. (10M) (CO2) (BLT4) a. It is a nice night.
   b. This crap game is over a garage in Fifty-second Street. . .
   c. . . .Nobody ever takes the newspapers she sells . . .
   d. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.
   e. . . . I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of, . . .
   f. When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

Question 1: Introduction to Bigram Language Models

A bigram language model estimates the probability of a word from the word immediately before it. It is a first-order Markov model because it uses only one previous token as context. The model is useful for estimating sentence likelihood, predicting the next word, and describing local word-order patterns.

Figure 4.1 gives observed bigram counts for eight words from the Berkeley Restaurant Project corpus. The vocabulary size is V = 1446, although only eight words are displayed in the table. The zero entries are important: an unseen bigram should not automatically receive probability zero when smoothing is applied.

The task is to perform add-one, also called Laplace, smoothing. One is added to every bigram count, including counts that are zero. The denominator is also adjusted by adding the vocabulary size to the total count of all bigrams beginning with the context word.

Question 1: Add-One Smoothing Formula

For a context word wi and a next word wj, add-one smoothing is P_Laplace(wj|wi) = (C(wi,wj)+1)/(C(wi)+V). Here C(wi,wj) is the observed bigram count, C(wi) is the total number of bigrams whose first word is wi, and V is the vocabulary size.

The numerator increases every individual bigram count by one. The denominator increases the total context count by V because one is added to each of the V possible next words. This ensures that the probabilities across the complete vocabulary sum to one for a given context.

The denominator is therefore row_total + 1446. It is not simply the row_total + 1. This is a frequent mistake in add-one smoothing. The smoothing constant is one for each possible next-word type, so the total added mass equals the vocabulary size.

Question 1: Calculating Row Totals

The first step is to add all counts across each row. For the row i, the total is 5+827+0+9+0+0+0+2 = 843. This is the number of observed continuations represented in the displayed row, not the total count of every possible continuation in the full vocabulary.

For want, the displayed row total is 2+0+608+1+6+6+5+1 = 629. For to, the total is 2+0+4+686+2+0+6+211 = 911. The same addition is performed for all remaining rows.

The row totals are: i=843, want=629, to=911, eat=62, chinese=84, food=35, lunch=4, and spend=2. Adding V=1446 gives the smoothed denominator for each row.

Question 1: Denominators for Every Context Word

For context i, the denominator is 843+1446=2289. For want, it is 629+1446=2075. For to, it is 911+1446=2357. These values are used for every probability in their respective rows.

For eat, the denominator is 62+1446=1508. For chinese, it is 84+1446=1530. For food, it is 35+1446=1481. For lunch, it is 4+1446=1450, and for spend, it is 2+1446=1448.

Notice that the denominator depends on the previous word, not on the next word. All probabilities in a particular row share the same denominator, while their numerators vary according to the bigram count plus one.

Question 1: Worked Calculation: P(want | i)

The observed count C(i,want) is 827. The total count C(i) is 843, and the vocabulary size is 1446. Substituting these values into the formula gives P(want|i)=(827+1)/(843+1446).

The numerator is 828 and the denominator is 2289. Therefore P(want|i)=828/2289, which is approximately 0.361730. This is a relatively large probability because “i want” is a frequent bigram in the restaurant-domain corpus.

The smoothing adjustment has a small effect on a large observed count. It changes 827 to 828 and adjusts the denominator by the full vocabulary size. The probability remains substantial, but it is lower than the unsmoothed relative frequency 827/843.

Calculation reminder

For every cell, use the count from the figure, add one to the numerator, and divide by the context-row total plus the full vocabulary size V=1446.

Question 1: Worked Calculation: P(eat | to)

The count of the bigram “to eat” is 686. The row total for to is 911, and V=1446. The add-one probability is (686+1)/(911+1446)=687/2357.

Dividing gives approximately 0.291472. This is another high probability because the phrase “to eat” is common in restaurant-related language. The smoothed result is positive and properly normalized with all other possible next words in the full vocabulary.

The same method applies to any cell. First locate the count at the intersection of the context row and the next-word column. Then add one to that count and divide by the row total plus 1446.

Question 1: Worked Calculation for a Zero Count

Consider the bigram “i chinese”. Its observed count is zero. Add-one smoothing changes the numerator from zero to one. The denominator for the i row is 2289, so P(chinese|i)=1/2289, approximately 0.000437.

This illustrates the main purpose of smoothing. Without smoothing, the maximum-likelihood estimate would be zero, and any sentence containing this unseen bigram would receive a zero probability under the bigram model.

Laplace smoothing does not claim that an unseen bigram is common. It assigns a small non-zero probability so that unseen events remain possible. This is especially useful when a language model is applied to text that differs slightly from its training corpus.

Calculation reminder

For every cell, use the count from the figure, add one to the numerator, and divide by the context-row total plus the full vocabulary size V=1446.

Question 1: Worked Calculation: Context want

The row total for want is 629, so its smoothed denominator is 2075. The count of “want to” is 608, giving P(to|want)=(608+1)/2075=609/2075, approximately 0.293494.

The count of “want chinese” is six, so P(chinese|want)=7/2075, approximately 0.003373. The count of “want want” is zero, so P(want|want)=1/2075, approximately 0.000482.

These calculations show how smoothing preserves the relative frequency structure. Frequent continuations retain larger probabilities, while zero-count continuations receive a small shared probability mass.

Question 1: Worked Calculation: Context to

The to row has total count 911 and denominator 2357. Its largest displayed continuation is eat with count 686. The smoothed probability is 687/2357, approximately 0.291472.

The count for “to spend” is 211, so P(spend|to)=212/2357, approximately 0.089945. The count for “to food” is zero, so P(food|to)=1/2357, approximately 0.000424.
Even when the observed count is large, the probability is not calculated by dividing by the sum of only the eight displayed counts plus eight. The complete vocabulary size is part of the stated problem and must be retained.

Calculation reminder

For every cell, use the count from the figure, add one to the numerator, and divide by the context-row total plus the full vocabulary size V=1446.

Question 1: Worked Calculation: Context eat

The row total for eat is 62, giving a denominator of 1508. The displayed counts include 16 for “eat chinese”, two for “eat food”, and 42 for “eat lunch”.
Therefore P(chinese|eat)=17/1508, approximately 0.011273. P(food|eat)=3/1508, approximately 0.001989. P(lunch|eat)=43/1508, approximately 0.028515.
The remaining zero-count cells in this row each receive probability 1/1508, approximately 0.000663. A single smoothing rule is applied consistently to all eight cells.

Question 1: Worked Calculation: Context chinese

The chinese row total is 84, and its denominator is 1530. The displayed counts are one for “chinese i”, 82 for “chinese food”, and one for “chinese lunch”. All other displayed bigrams in this row have count zero.
P(food|chinese)=(82+1)/1530=83/1530, approximately 0.054248. P(i|chinese)=2/1530, approximately 0.001307. P(lunch|chinese)=2/1530, also approximately 0.001307.
The zero-count cells receive 1/1530, approximately 0.000654. This is a small probability, but it avoids assigning an impossible value to an unseen continuation.

Calculation reminder

For every cell, use the count from the figure, add one to the numerator, and divide by the context-row total plus the full vocabulary size V=1446.

Question 1: Worked Calculation: Context food

The food row total is 35, so its denominator is 1481. The observed counts are 15 for “food i”, 15 for “food to”, one for “food chinese”, and four for “food food”.
P(i|food)=16/1481, approximately 0.010803. P(to|food)=16/1481, also approximately 0.010803. P(food|food)=5/1481, approximately 0.003376.
The remaining zero-count continuations each receive 1/1481, approximately 0.000675. This row demonstrates that even relatively small observed counts become positive smoothed estimates.

Question 1: Worked Calculation: Context lunch and spend

The lunch row total is four, so its denominator is 1450. The counts are two for “lunch i”, one for “lunch chinese”, and one for “lunch food”. The corresponding smoothed probabilities are 3/1450, 2/1450, and 2/1450.
The spend row total is two, so its denominator is 1448. The counts are one for “spend i” and one for “spend to”. Each corresponding smoothed probability is 2/1448, approximately 0.001381.
All other displayed zero-count continuations from lunch or spend receive 1/1450 or 1/1448 respectively. These low-count contexts show how add-one smoothing has a proportionally larger effect when the training count is small.

Calculation reminder

For every cell, use the count from the figure, add one to the numerator, and divide by the context-row total plus the full vocabulary size V=1446.

Question 1: Complete Smoothed Probability Matrix

The following table gives all 64 requested values for the eight displayed words. Each value is calculated from the corresponding count plus one, divided by the row total plus 1446. Values are rounded to six decimal places.

Add-one smoothed conditional probabilities: P(wj | wi) = (C(wi,wj)+1)/(C(wi)+V), V=1446.

Row totals: i=843, want=629, to=911, eat=62, chinese=84, food=35, lunch=4, spend=2

	i	want	to	eat	chinese	food	lunch	spend
i	0.002621	0.361730	0.000437	0.004369	0.000437	0.000437	0.000437	0.001311
want	0.001446	0.000482	0.293494	0.000964	0.003373	0.003373	0.002892	0.000964
to	0.001273	0.000424	0.002121	0.291472	0.001273	0.000424	0.002970	0.089945
eat	0.000663	0.000663	0.001989	0.000663	0.011273	0.001989	0.028515	0.000663
chinese	0.001307	0.000654	0.000654	0.000654	0.000654	0.054248	0.001307	0.000654
food	0.010804	0.000675	0.010804	0.000675	0.001350	0.003376	0.000675	0.000675
lunch	0.002069	0.000690	0.000690	0.000690	0.001379	0.001379	0.000690	0.000690
spend	0.001381	0.000691	0.001381	0.000691	0.000691	0.000691	0.000691	0.000691

The table should be read row-wise as a conditional probability distribution. For example, the row labelled i contains P(i|i), P(want|i), P(to|i), and so on. The row labelled want contains the probabilities of each possible next word after want.

Question 1: Why Zero Counts Become Non-Zero

Maximum-likelihood bigram estimates assign probability C(wi,wj)/C(wi). When C(wi,wj)=0, the estimate is zero. This can be problematic because a sentence probability is calculated by multiplying conditional probabilities; one zero factor makes the entire sentence probability zero. Add-one smoothing replaces every count c with c+1. Therefore, a zero count becomes one. The denominator is increased by V so that the total probability over the complete vocabulary remains one for each context.

The method is simple and easy to calculate by hand. However, it can over-smooth language data because it assigns equal pseudo-count one to every vocabulary item, even though many continuations are extremely unlikely.

Calculation reminder

For every cell, use the count from the figure, add one to the numerator, and divide by the context-row total plus the full vocabulary size V=1446.

Question 1: Normalization Check

For a fixed context wi, the smoothed probabilities over all V vocabulary words sum to one. The numerator sum is the sum of all observed counts in that row plus V. The denominator is exactly the same quantity: C(wi)+V.

Thus the complete distribution is normalized. The displayed eight columns alone generally sum to less than one because the other 1438 vocabulary items are not shown in Figure 4.1. Their probability mass is still included in the denominator.

This point explains why summing only the eight displayed probabilities does not have to produce one. The matrix is a subset of the full conditional distribution, while the vocabulary size refers to the complete corpus vocabulary.

Question 1: Limitations and Alternative Smoothing

Laplace smoothing is often useful as a teaching example, but it may assign too much probability to unseen events in a large vocabulary. In natural-language applications, methods such as Good-Turing discounting, backoff, and interpolated Kneser-Ney smoothing often perform better.

Backoff methods use lower-order probabilities when a higher-order n-gram is unseen. Interpolation combines probabilities from unigram, bigram, and sometimes trigram models. These methods use more information about the distribution of words.

Despite its limitations, add-one smoothing is valuable because the calculation is transparent. It demonstrates the general principle that probability estimates should account for events not observed in a finite training sample.

Question 1: Applications of Smoothed Bigram Probabilities

Smoothed bigram probabilities can be used to score a sentence. The probability of a sequence is approximated by the product of the conditional probability of each word given the previous word, together with an initial-word probability.

In speech recognition, a language model helps select a plausible word sequence among alternatives produced by an acoustic model. In spelling correction and text prediction, bigram probabilities help rank likely continuations.

In a restaurant dialogue system, phrases such as “i want”, “to eat”, “chinese food”, and “food lunch” may provide domain-specific evidence. The corpus counts reflect the patterns that users frequently produce in that application.

Question 1: Conclusion and Final Answer for Question 1

Add-one smoothing uses $P(w_j|w_i)=(C(w_i,w_j)+1)/(C(w_i)+1446)$. The row totals are i=843, want=629, to=911, eat=62, chinese=84, food=35, lunch=4, and spend=2.

The smoothed denominators are 2289, 2075, 2357, 1508, 1530, 1481, 1450, and 1448 respectively. Every cell in the matrix is obtained by adding one to the original count and dividing by its row denominator.

The complete 8×8 matrix is included on the preceding calculation page. All zero counts receive a positive probability. This completes the add-one smoothing of the displayed bigram counts.

Question 2: Introduction to Bag-of-Words Generation

A bag of words is a collection of words in which the original order has been removed. It preserves which words occur and how many times each word occurs, but it does not preserve their positions. The goal of bag generation is to reconstruct a plausible sentence order from this unordered collection.

This task is challenging because many different sentences can contain the same words. For example, “the dog chased a cat” and “a cat chased the dog” contain similar word inventories but express different meanings. A language model can rank possible orders according to how natural they are.

An N-gram language model provides a practical solution. It estimates the probability of a word from the previous $N-1$ words. A bigram model uses one previous word, while a trigram model uses the previous two words. Higher-order context generally captures more local structure.

Question 2: System Architecture

The proposed system has four main stages: read the bag of words, train or load an N-gram model, generate candidate sequences, and score each candidate. The candidate with the highest language-model score is returned as the guessed original order.

The training corpus should contain sentences written in the same language and preferably in a similar domain as the bag-of-words input. The corpus is tokenized, normalized, and converted into N-gram counts. Start and end markers are added so the model can learn sentence beginnings and endings.

The generator can use greedy decoding, beam search, or stochastic sampling. Greedy decoding chooses the most likely next word at every step, while beam search keeps several partial candidates and is less likely to make an early locally optimal choice that harms the full sentence.

Question 2: N-Gram Model Definition

An N-gram is a sequence of N consecutive tokens. A unigram contains one word, a bigram contains two words, and a trigram contains three words. An N-gram language model estimates $P(w_i|w_{i-N+1},...,w_{i-1})$.

For a bigram model, the conditional probability is estimated as count(previous,current)/count(previous), with smoothing added for unseen events. For a trigram model, the context consists of two previous words. The model therefore captures more local grammatical and lexical context.

The probability of a complete sentence is approximated by multiplying the conditional probabilities of its words. In practice, logarithms are used to avoid numerical underflow. The log probability is the sum of the log conditional probabilities.

Question 2: Preparing a Training Corpus

The training corpus is a collection of sentences. Each sentence should be normalized consistently: convert to lowercase if case is not important, tokenize punctuation, and remove or preserve punctuation according to the task. Add a start token and an end token to represent sentence boundaries.

Training and testing data should be separated when evaluating the model. If the test sentence is also included in training, the model may memorize it and produce an overly optimistic result. A general-domain corpus can be used for general sentences, while a restaurant corpus is better for restaurant-related language.

The vocabulary should include an unknown-word token for words that do not occur in the training corpus. This prevents the model from assigning an impossible probability to an unseen input token.

Question 2: Representing the Bag of Words

The input bag is represented as a multiset rather than a simple set. A multiset preserves repeated occurrences. For example, the bag for “the cat saw the dog” contains two occurrences of the and one occurrence each of cat, saw, and dog.

The generator maintains a remaining-count dictionary. When a word is selected for the next position, its remaining count is reduced by one. A word cannot be selected again after its count reaches zero. This guarantees that the generated sequence contains exactly the input words.

Special tokens such as sentence-start and sentence-end markers are not part of the original bag. They are added internally for language-model scoring and removed from the final displayed sentence.

Question 2: Candidate Generation with Greedy Search

Greedy decoding begins with the start token. At each step, it considers every word that still has a positive remaining count and selects the word with the highest conditional probability given the current context.

After selecting a word, the algorithm appends it to the partial sequence and reduces its remaining count. The process continues until every input word has been used. The end token can then be included in the final score.

Greedy decoding is efficient, but it may make a locally good choice that prevents a better complete sequence. For example, the best immediate continuation may create an awkward phrase later. Beam search addresses this by retaining multiple partial sequences.

Implementation note

The candidate sequence must consume every token exactly once. Use a multiset or Counter rather than a set so repeated words are preserved.

Question 2: Beam Search for Better Ordering

Beam search keeps the best B partial sequences at each generation step. For every partial sequence, it expands all available next-word choices, calculates their updated log probabilities, and ranks the expanded candidates.

Only the top B candidates are retained for the next step. When all words have been used, the algorithm adds the end-of-sentence probability and selects the highest-scoring complete sequence.

A larger beam width can improve search quality but requires more computation. A small beam is often sufficient for short bags. The search still cannot guarantee recovery of the exact original order because the bag has discarded information.

Question 2: Python Program: Corpus Preprocessing

The following program uses Python and NLTK-style tokenization can be added if required. To keep the example self-contained, it uses simple whitespace tokenization and lowercasing. The training corpus is supplied as a list of sentences.

Code:

```
import re
from collections import Counter, defaultdict

corpus = [
"i want to eat chinese food",
"i want to spend lunch",
"chinese food is good",
"we want to eat lunch"
]

def tokenize(text):
return re.findall(r"[a-z]+", text.lower())
```

sentences = [tokenize(s) for s in corpus]
The same tokenizer must be applied to the bag-of-words input. In a production implementation, punctuation and contractions should be handled more carefully, and the training corpus should be large enough to represent the target language.

Question 2: Python Program: Building Bigram Counts

After tokenization, the model counts adjacent word pairs. A start token is inserted before each sentence and an end token is appended after it. The bigram counts record how often a word follows a particular previous word.

Code:

```
bigram = defaultdict(Counter)
unigram = Counter()
for sent in sentences:
tokens = ["<s>"] + sent + ["</s>"]
for w in tokens:
unigram[w] += 1
for a, b in zip(tokens, tokens[1:]):
bigram[a][b] += 1

V = len(unigram) + 1
```

The vocabulary size is used for add-one smoothing. For a previous word a and candidate word b, the smoothed bigram probability is (count(a,b)+1)/(count(a)+V). This assigns a non-zero probability to unseen transitions.

Question 2: Python Program: Bigram Probability Function

The probability function returns the smoothed conditional probability of a candidate next word given the previous word. A small additive constant avoids zero probabilities. In this example, Laplace smoothing uses one.

Code:

```
def bigram_prob(previous, current):
numerator = bigram[previous][current] + 1
denominator = unigram[previous] + V
return numerator / denominator

import math
def log_bigram_prob(previous, current):
return math.log(bigram_prob(previous, current))
```

Log probabilities are used because multiplying many small probabilities can underflow to zero in floating-point arithmetic. Adding log probabilities gives the same ranking as multiplying probabilities, while remaining numerically stable.

Implementation note

The candidate sequence must consume every token exactly once. Use a multiset or Counter rather than a set so repeated words are preserved.

Question 2: Python Program: Greedy Bag Generator

The generator takes a list of words and repeatedly selects the remaining word with the highest bigram probability. It uses a Counter so duplicate words are retained and cannot be lost.

Code:

```
def generate_greedy(bag):
remaining = Counter(tokenize(" ".join(bag)))
result = []
previous = "<s>"
while sum(remaining.values()) > 0:
candidates = [w for w, n in remaining.items() if n > 0]
best = max(candidates,
key=lambda w: bigram_prob(previous, w))
result.append(best)
remaining[best] -= 1
previous = best
return " ".join(result)
```

This basic function produces a valid permutation of the input words. It does not use a sentence-end score during selection, and it uses only the previous word. It is a simple baseline that can be improved with beam search or a trigram model.

Question 2: Python Program: Trigram Model

A trigram model conditions each word on the previous two words. It can distinguish contexts that a bigram model treats as identical. For example, the probability of “food” may differ depending on whether the preceding context is “chinese” or “good”.

Code:

```
trigram = defaultdict(Counter)
context_count = Counter()
for sent in sentences:
tokens = ["<s>", "<s>"] + sent + ["</s>"]
for a, b, c in zip(tokens, tokens[1:], tokens[2:]):
trigram[(a, b)][c] += 1
context_count[(a, b)] += 1

def trigram_prob(a, b, c):
return (trigram[(a, b)][c] + 1) / (context_count[(a, b)] + V)
```

The trigram model requires more training data because there are many more possible two-word contexts. Smoothing and backoff are important when the exact context has not appeared in the corpus.

Question 2: Beam Search Algorithm

Beam search starts with one empty candidate containing the start marker, a copy of the remaining word counts, and a score of zero. At each iteration, every candidate is expanded by appending each word that remains available.

The score is updated by adding the log conditional probability of the appended word. The remaining count for that word is reduced. All expansions are sorted by score, and only the best B are retained.

At the final step, the end-token probability is added. The candidate with the greatest total log probability is returned. Because the bag is small in many classroom examples, a beam width such as five or ten can be used.

Question 2: Evaluation and Expected Output

The generated sequence should be evaluated in more than one way. Exact match checks whether the output is identical to the original sentence. Word-order accuracy measures how many words appear in the correct positions. BLEU or edit distance can provide additional comparisons.

A useful test uses a known sentence, shuffles its words to create a bag, and asks the generator to reconstruct an order. The original sentence is retained only for evaluation and is not supplied to the algorithm.

Example input bag: {food, i, chinese, want, to, eat}. A plausible generated output is “i want to eat chinese food”. The exact output depends on the corpus and model probabilities. Other grammatically possible arrangements may receive similar scores.

Implementation note

The candidate sequence must consume every token exactly once. Use a multiset or Counter rather than a set so repeated words are preserved.

Question 2: Handling Repeated and Unknown Words

Repeated words must be stored with their frequencies. A Python set is unsuitable because it removes duplicates. Counter is appropriate because it maps each word to the number of times it appears in the bag.

Unknown words can be mapped to an <UNK> token for model scoring, while the original spelling is retained for output. If several unknown words occur, a system may need character-level or subword information to distinguish them.

Input validation should also check that the bag is not empty and that all items are valid tokens. The program should preserve the exact number of each word so that no input token is added or omitted.

Question 2: Limitations and Ambiguity

The original order cannot always be recovered uniquely. A bag of words removes syntax, word position, punctuation, and sentence boundaries. Multiple sentences may have exactly the same bag, so a language model can only guess the most probable ordering under its training distribution.

A language model may generate a fluent sentence that is not the original sentence. This is not necessarily a coding error; it is a consequence of missing information. The output should therefore be described as a best guess rather than a guaranteed reconstruction.

Domain mismatch is another limitation. A model trained on restaurant dialogues may perform poorly on medical text or informal social media. The training corpus should resemble the expected input domain.

Question 2: Improving the Model

Possible improvements include using trigram or four-gram models, beam search, interpolation, Kneser-Ney smoothing, and part-of-speech constraints. A syntax-aware decoder can reject candidates that violate basic grammatical rules.

Semantic information can also help. If the bag contains a subject, verb, and object, a semantic-role model may prefer orders that produce plausible relations. A neural language model can provide a stronger sentence-level score than a simple N-gram model.
Despite these improvements, no model can guarantee the original order from an unordered bag alone. The best result is a plausible sentence that maximizes the chosen model score.

Question 2: Conclusion for Question 2

The bag-generation task can be approached using an N-gram language model trained on a corpus. The program preserves word counts, generates candidate permutations, and ranks them according to conditional probabilities.
A bigram model is simple and efficient, while a trigram model uses more context. Greedy decoding provides a baseline, and beam search explores multiple candidates to reduce the risk of a poor early choice.
The method produces a best-guess sentence, not a guaranteed reconstruction. The original word order is lost when the sentence is converted to a bag. Corpus quality, smoothing, vocabulary handling, and search strategy strongly influence the result.

Implementation note

The candidate sequence must consume every token exactly once. Use a multiset or Counter rather than a set so repeated words are preserved.

Question 2: Complete Program Workflow and Sample Run

The complete workflow begins by reading a training corpus and tokenizing every sentence. The model counts unigrams and N-grams, calculates smoothed conditional probabilities, and receives a bag as a list of tokens. The decoder then searches for a high-probability ordering without changing the multiset of input words.
Example bag: ["food", "i", "chinese", "want", "to", "eat"]. One plausible output is “i want to eat chinese food”. The exact output depends on the corpus and the search method. The program should print both the generated sequence and its log probability so the result can be inspected.
A useful verification step compares Counters for the input and output. If Counter(input_words) equals Counter(output_words), the generator has used every input word exactly the required number of times. This test catches accidental duplication or omission.

Question 2: Final Discussion and Summary of Bag Generation

Bag generation is an inverse ordering problem. The input contains lexical items but no order, and the N-gram model supplies statistical knowledge about likely local sequences. The generator combines these two sources of information to produce a plausible sentence.
The bigram model is straightforward to train and score, while trigram models provide richer context. Greedy decoding is fast, and beam search generally explores better alternatives. Smoothing is needed to handle unseen N-gram transitions.
The original sentence cannot always be recovered uniquely because the bag representation discards information. The correct description of the output is therefore a best guess according to the language model, not a guaranteed reconstruction.

Question 3: Introduction to Penn Treebank POS Tagging

Part-of-speech tagging assigns a grammatical category to each word in a sentence. The Penn Treebank tagset is a widely used inventory of labels for nouns, verbs, adjectives, adverbs, pronouns, determiners, prepositions, conjunctions, and other categories.

The tag of a word depends on its function in context, not only on its dictionary meaning. For example, “over” may be a preposition or an adverb, and “putting” may be a verb form or a noun-like gerund. The surrounding words help determine the correct tag.
This solution uses Penn Treebank tags and ignores punctuation as permitted. Each sentence is shown as a sequence of word/tag pairs, followed by an explanation of the more difficult decisions.

Question 3: Penn Treebank Tags Used in the Answers

Common noun tags include NN for singular common nouns and NNS for plural common nouns. Proper nouns use NNP for singular proper names and NNPS for plural proper names. Adjectives use JJ, JJR, and JJS for positive, comparative, and superlative forms.
Verb tags include VB for base form, VBD for past tense, VBG for present participle or gerund, VBN for past participle, VBP for non-third-person singular present, and VBZ for third-person singular present.
Other frequent tags include DT for determiner, PRP for personal pronoun, PRP$ for possessive pronoun, IN for preposition or subordinating conjunction, TO for the infinitival marker to, CC for coordinating conjunction, RB for adverb, WRB for wh-adverb, and WDT for wh-determiner.

Question 3: Sentence (a): It is a nice night

Sentence: “It is a nice night.” The correct sequence is It/PRP is/VBZ a/DT nice/JJ night/NN.
The word It is a personal pronoun and functions as the subject of the clause.
The word is is the third-person singular present form of the verb be, so it receives VBZ. The word a is an article and is tagged DT. The word nice describes the noun night and is therefore an adjective, JJ.
Night is a singular common noun and functions as the head of the noun phrase “a nice night”. It receives NN. This short sentence illustrates a basic subject–verb–complement structure.

Question 3: Sentence (b): This crap game is over a garage

Sentence: “This crap game is over a garage in Fifty-second Street.” The sequence is This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP.
This is a demonstrative determiner modifying the noun phrase “crap game”. Crap functions as a noun adjunct modifying game, so NN is appropriate. Game is the head singular noun. Is is VBZ. Over is treated as IN because it introduces the prepositional phrase “over a garage”. A is DT, garage is NN, and in is IN. Fifty-second is treated as a compound ordinal modifier, JJ, and Street is part of the proper street name, NNP.

Tagging check

Read the sentence from left to right and verify that every word has exactly one Penn Treebank tag. The selected tag should agree with the word's grammatical function in its sentence.

Question 3: Sentence (b): Noun Compounds and Proper Names

Noun compounds often contain a noun modifying another noun. In “crap game”, crap describes the type of game and remains a noun in the Penn Treebank convention. The head noun is game. This differs from an ordinary adjective such as nice.
The expression “Fifty-second Street” is a street name. A practical tagger may tokenize the hyphenated ordinal differently depending on preprocessing. If it remains one token, JJ is a reasonable tag because it modifies the proper noun Street.

Street is capitalized as part of a named location and is therefore NNP. If the phrase were a generic reference to a street rather than part of a name, a common-noun tag NN could be appropriate.

Question 3: Sentence (c): Nobody ever takes the newspapers

Sentence: “Nobody ever takes the newspapers she sells.” The sequence is Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ.

Nobody is an indefinite pronoun. The Penn Treebank tagset commonly assigns NN to nobody in this construction. Ever modifies the verb takes and is an adverb, RB.

Takes is the third-person singular present form of take, so it is VBZ. The is a determiner, DT. Newspapers is a plural common noun, NNS. She is a personal pronoun, PRP, and sells is a third-person singular present verb, VBZ.

Question 3: Sentence (c): Relative Clause Analysis

The clause “she sells” modifies newspapers and functions as a relative clause with the relative pronoun omitted. The subject she is PRP and the verb sells is VBZ. The object of sells is understood from the noun phrase newspapers.

The word ever is not an adjective because it does not describe a noun. It modifies the frequency or generality of the action and therefore receives RB. This is a useful example of how syntactic position helps determine a tag.

Nobody is singular in grammatical agreement, which explains the third-person singular form takes. The sentence therefore contains a subject–verb agreement relation between Nobody and takes.

Tagging check

Read the sentence from left to right and verify that every word has exactly one Penn Treebank tag. The selected tag should agree with the word's grammatical function in its sentence.

Question 3: Sentence (d): Adjectives Before a Noun

Sentence: “He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.” The sequence is He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN.

He is a personal pronoun, PRP. Is is VBZ. A is DT. Tall and skinny describe guy and are adjectives, JJ. Guy is a singular common noun, NN.

With introduces a prepositional phrase and is IN. Long, sad, and mean-looking all modify kisser, so each receives JJ. Kisser is NN. And is a coordinating conjunction, CC, connecting the noun phrase descriptions.

Question 3: Sentence (d): Compound Adjectives and Coordination

Mean-looking is a compound adjective. It describes the appearance or quality of kisser, so it is tagged JJ. The hyphen remains part of the word if the tokenizer keeps the compound as one token.

Mournful modifies voice and is JJ. Voice is the head singular noun and is NN. The second noun phrase is coordinated with the preceding description by and, which is tagged CC.

The comma is ignored as requested. If punctuation were retained, the comma would receive a punctuation tag such as comma, but it is not included in the present word/tag sequence.

Question 3: Sentence (e): First Clause and Gerund

Sentence begins: “I am sitting in Mindy’s restaurant putting on the gefillte fish...” The sequence is I/PRP am/VBP sitting/VBG in/IN Mindy’s/NNP restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN.

I is a personal pronoun, PRP. Am is the non-third-person singular present form of be and receives VBP. Sitting is the present participle of sit in the progressive construction “am sitting”, so it is VBG.

In introduces a prepositional phrase and is IN. Mindy’s is a possessive proper name. When tokenized as one possessive form, it may be tagged NNP; a tokenizer that separates the possessive clitic may tag the name NNP and the clitic POS.

Tagging check

Read the sentence from left to right and verify that every word has exactly one Penn Treebank tag. The selected tag should agree with the word's grammatical function in its sentence.

Question 3: Sentence (e): Phrasal Verb and Food Term

Putting on is interpreted as a phrasal verb in the context of serving or eating food. Putting is VBG because it is the present participle in the ongoing action. On functions as a particle and is tagged RP in this phrasal-verb reading.

The is a determiner, DT. Gefillte is a descriptive food adjective and is tagged JJ. Fish is a singular mass/common noun in this context and is tagged NN.

Specialized food words may not be present in a general training lexicon. The tagger must use context: gefillte appears before fish and describes the type of food, making JJ a reasonable tag.

Question 3: Sentence (e): Relative Clause and Complement

The next clause is “which is a dish I am very fond of”. Which introduces a relative clause and is tagged WDT. Is is VBZ because its subject is the singular relative pronoun which.

A is DT and dish is NN. I is PRP. Am is VBP because the subject is first-person singular. Very modifies the adjective fond and is RB. Fond is JJ.

Of is IN and completes the adjective phrase “fond of”. This preposition is selected by the adjective fond. The sequence therefore illustrates a prepositional complement of an adjective.

Question 3: Sentence (f): When-Clause and Subjects

Sentence begins: “When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.” The sequence begins When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG.

When introduces a temporal subordinate clause and is tagged WRB. A is DT. Guy is NN. And is CC. A is DT, doll is NN, and get is VBP because the coordinated subject “a guy and a doll” is plural.

To is the infinitival marker or particle introducing the following verb form and is tagged TO. Taking is VBG. The construction “get to taking” is a colloquial expression in which taking is a gerund-like participial form.

Tagging check

Read the sentence from left to right and verify that every word has exactly one Penn Treebank tag. The selected tag should agree with the word's grammatical function in its sentence.

Question 3: Sentence (f): Object and Directional Adverbs

Peeks is the plural noun functioning as the object of taking, so it receives NNS. Back and forth form a directional or manner expression. Back is RB and forth is RB in this adverbial use.
At introduces a prepositional phrase and receives IN. Each is a determiner, DT, modifying other. Other is tagged JJ in the phrase “each other” in this analysis, although taggers may treat the expression as a fixed reciprocal construction.
The phrase “each other” is a reciprocal expression. Its two words work together as the object of at. Context is important because each can be a pronoun in some environments, but here it modifies other.

Question 3: Sentence (f): Discourse Markers and Main Clause

Why is used as a discourse or resultative marker in the colloquial construction and receives WRB. There is an adverbial or presentational use and is tagged RB, not the existential pronoun there.
You is a personal pronoun, PRP. Are is the non-third-person singular present form of be and receives VBP. Indeed modifies the assertion and is an adverb, RB.
The main clause “there you are indeed” is an idiomatic expression. A syntactic tagger may differ on the precise role of why and there, but WRB and RB are reasonable Penn Treebank choices in this context.

Question 3: Complete Tagging Table for Sentences (a)–(c)

Sentence (a): It/PRP is/VBZ a/DT nice/JJ night/NN.
Sentence (b): This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP.
Sentence (c): Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ.
These first three sentences include pronouns, determiners, adjectives, common nouns, proper nouns, adverbs, verbs, and prepositions. The labels follow the Penn Treebank conventions and ignore punctuation as the question permits.

Tagging check

Read the sentence from left to right and verify that every word has exactly one Penn Treebank tag. The selected tag should agree with the word's grammatical function in its sentence.

Question 3: Complete Tagging Table for Sentences (d)–(f)

Sentence (d): He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN.
Sentence (e): I/PRP am/VBP sitting/VBG in/IN Mindy’s/NNP restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN.
Sentence (f): When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/WRB there/RB you/PRP are/VBP indeed/RB.

Ambiguous and Difficult Tagging Decisions

Some words are difficult because they can belong to multiple parts of speech. Over may be an adverb, adjective, or preposition. In sentence (b), it introduces the phrase “over a garage”, so IN is selected.

Putting on may be analyzed as a verb plus particle, with on tagged RP. If the context were purely spatial, on could be a preposition IN. The phrase-level meaning determines the most appropriate tag.

Fifty-second is a hyphenated ordinal modifier. Treating it as JJ is reasonable when it modifies Street as part of a proper street name. Tokenization conventions may split it into separate tokens, which can alter the exact tag sequence.

Question 3: Why Context Matters in POS Tagging

A word's part of speech is determined by its syntactic function. In “a nice night”, nice is an adjective because it modifies a noun. In another sentence, a related form could be used as a noun or verb.

Penn Treebank tagging provides a consistent label inventory, but some constructions remain ambiguous. Idioms, colloquial speech, proper names, and domain-specific vocabulary may lead different annotators or automatic taggers to choose different labels.

The goal in this exercise is to provide a defensible context-based tag for every word. Where a tokenization or lexical convention may vary, the chosen assumption is explained so that the analysis is transparent.

Tagging check

Read the sentence from left to right and verify that every word has exactly one Penn Treebank tag. The selected tag should agree with the word's grammatical function in its sentence.

Question 3: Conclusion and Final POS Tag Summary

The Penn Treebank tagset represents grammatical categories with compact labels such as NN, NNS, NNP, JJ, RB, PRP, DT, IN, CC, VBG, VBP, VBZ, TO, WDT, and WRB. The six sentences have been tagged word by word.

The most difficult decisions concern noun compounds, proper names, phrasal particles, gerunds, reciprocal expressions, and discourse markers. These decisions require context rather than isolated dictionary lookup.

In conclusion, POS tagging is a foundational NLP task. Accurate tags support parsing, information extraction, text analysis, and language-model construction. The supplied tags provide a consistent Penn Treebank-style analysis of all six sentences.